```
Y = 'a root-level constant'

module M
  Y = 'a constant in M'
  Y      # => "a constant in M"
  ::Y    # => "a root-level constant"
end
```

The Module class also provides an instance method and a class method that, confusingly, are both called constants. Module#constants returns all constants in the current scope, like your file system's ls command (or dir command, if you're running Windows). Module.constants returns all the top-level constants in the current program, including class names:

```
M.constants                          # => [:C, :Y]
Module.constants.include? :Object    # => true
Module.constants.include? :Module    # => true
```

Finally, if you need the current path, check out Module.nesting:

```
module M
  class C
    module M2
      Module.nesting    # => [M::C::M2, M::C, M]
    end
  end
end
```

The similarities between Ruby constants and files go even further: you can use modules to organize your constants, the same way that you use directories to organize your files. Let's look at an example.

The Rake Example

The earliest versions of Rake, the popular Ruby build system, defined classes with obvious names, such as Task and FileTask. These names had a good chance of clashing with other class names from different libraries. To prevent clashes, Rake switched to defining those classes inside a Rake module:

gems/rake-0.9.2.2/lib/rake/task.rb

```
module Rake
  class Task
    # ...
```

Now the full name of the Task class is Rake::Task, which is unlikely to clash with someone else's name. A module such as Rake, which only exists to be a container of constants, is called a *Namespace*.

Spell: Namespace